Análise dos Repasses da Arrecadação Federal

Problemas e Fontes

Contexto

> **A arrecadação federal movimenta trilhões de reais todos os anos, mas depois que esses recursos são arrecadados, para onde parte desse dinheiro é destinada?**

Dentro desse universo, existe uma categoria específica de recursos: as **contribuições destinadas a outras entidades e fundos**, conhecidos como **"terceiros"**.

Esses valores são arrecadados por meio de documentos como:

- **DARF** — Documento de Arrecadação de Receitas Federais
- **GPS** — Guia da Previdência Social

Objetivo da análise

Neste estudo, vamos analisar especificamente esses **repasses**, buscando entender:

- **Quanto** é destinado a essas entidades;
- **Quais instituições** recebem os maiores valores;
- **Como os repasses evoluem** ao longo do tempo;
- **Qual a representatividade financeira** desses repasses em relação à arrecadação federal;
- **Quais instituições possuem maior relevância financeira** dentro dessa categoria.

Dados utilizados

Para esta análise, serão utilizados **dados públicos catalogados pelo Governo Federal em 05 de agosto de 2022**.

Com um pouco mais de 2000 registros. Esse CSV carrega algumas informações sobre data, valores e informações da entidade.

Informação	Período
Início dos registros	**Janeiro de 2015**
Último registro	**Agosto de 2026**
Catalogação	**05 de agosto de 2022**
Categoria	**Outras entidades e fundos — "Terceiros"**

A partir desses dados, será possível analisar tanto a **distribuição dos recursos entre as entidades** quanto sua **evolução ao longo do tempo**, identificando quais instituições apresentam maior relevância financeira dentro dessa categoria de repasses.

O dado que vamos utilizar está armazenado em um link do drive, porém, foi retirado exatamente no link abaixo.

link: https://dados.gov.br/dados/conjuntos-dados/repasses-da-arrecadacao-federal

Ingestão dos dados

###Importando o arquivo

In [1]:
```
import pandas as pd
import re

link =
"https://drive.google.com/file/d/1lLxFUzdouLUESomjn0tqO_jjOiG2V_UT/view?
usp=sharing"

file_id = re.search(r"/d/([a-zA-Z0-9_-]+)", link).group(1)

url = f"https://drive.google.com/uc?id={file_id}"

df = pd.read_csv(url, sep=";")

df.head()
```

Out[1]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
0	jan/15	FNDE	153173 / 15253	2.604.077.590,00
1	jan/15	INCRA	373001 / 37201	181.456.016,00
2	jan/15	SENAI	33.564.543/0001-90	219.119.330,00
3	jan/15	SESI	33.641.358/0001-52	313.370.139,00
4	jan/15	SENAC	33.469.172/0001-68	334.005.249,00

Qualidade dos dados

###Quantidade de linhas

In [2]:
```
df.count()
```

Out[2]:

	0
Mês / Ano de Referência	2380
Entidade	2380
UC/CNPJ	2380
Total Repassado	2380

dtype: int64

###Valores nulos

In [3]:
```
df_linhas_com_nulos = df[df.isna().any(axis=1)]
df_linhas_com_nulos
```

Out[3]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2380	NaN	NaN	NaN	NaN
2381	NaN	NaN	NaN	NaN
2382	NaN	NaN	NaN	NaN
2383	NaN	NaN	NaN	NaN
2384	NaN	NaN	NaN	NaN
2385	NaN	NaN	NaN	NaN
2386	NaN	NaN	NaN	NaN
2387	NaN	NaN	NaN	NaN
2388	NaN	NaN	NaN	NaN
2389	NaN	NaN	NaN	NaN
2390	NaN	NaN	NaN	NaN
2391	NaN	NaN	NaN	NaN
2392	NaN	NaN	NaN	NaN
2393	NaN	NaN	NaN	NaN
2394	NaN	NaN	NaN	NaN
2395	NaN	NaN	NaN	NaN
2396	NaN	NaN	NaN	NaN

###Registros duplicados

In [4]:
```
print("Linhas idênticas:", df.duplicated().sum())
chave = ["Mês / Ano de Referência", "Entidade"]
dup = df[df.duplicated(subset=chave, keep=False)]
dup.sort_values(chave)
```

```
Linhas idênticas: 16
```

Out[4]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2380	NaN	NaN	NaN	NaN
2381	NaN	NaN	NaN	NaN
2382	NaN	NaN	NaN	NaN
2383	NaN	NaN	NaN	NaN
2384	NaN	NaN	NaN	NaN
2385	NaN	NaN	NaN	NaN
2386	NaN	NaN	NaN	NaN
2387	NaN	NaN	NaN	NaN
2388	NaN	NaN	NaN	NaN
2389	NaN	NaN	NaN	NaN
2390	NaN	NaN	NaN	NaN
2391	NaN	NaN	NaN	NaN
2392	NaN	NaN	NaN	NaN
2393	NaN	NaN	NaN	NaN
2394	NaN	NaN	NaN	NaN
2395	NaN	NaN	NaN	NaN
2396	NaN	NaN	NaN	NaN

###Tipos de dados incorretos

In [5]:
```
base = df.dropna()
base.sample(8, random_state=1)
```

Out[5]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
877	abr/19	SENAR	37.138.245/0001-90	89.251.005,00
2086	mar/25	SENAT	73.471.963/0001-47	48.414.953,47
120	ago/15	INCRA	373001 / 37201	99.312.699,00
2266	fev/26	SESC	33.469.164/0001-11	785526796,3
107	jul/15	SESC	33.469.164/0001-11	357.769.318,00
1232	jan/21	DPC	852001 / 00001	21.364.259,00
1264	mar/21	SEBRAE	00.330.854/0001-45	293.937.886,00
1684	abr/23	INCRA	373001 / 37201	196.583.829,00

###Formatos inconsistentes

In [6]:
```
com_milhar = base["Total Repassado"].str.contains(r"\.\d{3}")
sem_milhar = ~com_milhar & (base["Total Repassado"] != "0")
print("Valores com separador de milhar:", com_milhar.sum())
print("Valores sem separador de milhar:", sem_milhar.sum())
base.loc[sem_milhar, "Mês / Ano de Referência"].unique()
```

```
Valores com separador de milhar: 2070
Valores sem separador de milhar: 176
```

Out[6]:
```
array(['out/25', 'nov/25', 'dez/25', 'jan/26', 'fev/26', 'mar/26',
       'abr/26', 'mai/26', 'jun/26', 'jul/26', 'ago/26'], dtype=object)
```

In [7]:
```
base[sem_milhar].head(10)
```

Out[7]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2193	out/25	FNDE	153173 / 15253	2950662651
2194	out/25	INCRA	373001 / 37201	252386984,9
2195	out/25	SENAI	33.564.543/0001-90	218030840,1
2196	out/25	SESI	33.641.358/0001-52	319516639,5
2197	out/25	SENAC	33.469.172/0001-68	429318140,3
2198	out/25	SESC	33.469.164/0001-11	739128752,9
2199	out/25	SEBRAE	00.330.854/0001-45	472771291,6
2201	out/25	DPC	852001 / 00001	23184616,26
2202	out/25	ANAC	122002 / 00001	24891430,8
2203	out/25	SENAR	37.138.245/0001-90	215406046,8

In [8]:
```
[repr(e) for e in base["Entidade"].unique()]
```

Out[8]:
```
["'FNDE'",
 "'INCRA'",
 "'SENAI'",
 "'SESI'",
 "'SENAC'",
 "'SESC'",
 "'SEBRAE'",
 "'EMBRATUR'",
 "'DPC'",
 "'ANAC'",
 "'SENAR'",
 "'SEST'",
 "'SENAT'",
 "'SESCOOP'",
 "'SDR'",
 "'APEX-BR'",
 "'ABDI'",
 "'FNDE '",
 "'APEX'"]
```

In [9]:
```
base["Entidade"].value_counts()
```

Out[9]:

Entidade	count
INCRA	140
SENAI	140
SESI	140
SEST	140
SENAC	140
SESC	140
SEBRAE	140
EMBRATUR	140
DPC	140
ANAC	140
SENAR	140
ABDI	140
SENAT	140
SESCOOP	140
SDR	140
FNDE	137
APEX-BR	132
APEX	8
FNDE	3

dtype: int64

In [10]:
```
apex = base[base["Entidade"].isin(["APEX", "APEX-BR"])]
apex.groupby("Entidade").agg(
    primeiro_mes=("Mês / Ano de Referência", "first"),
    ultimo_mes=("Mês / Ano de Referência", "last"),
    registros=("Mês / Ano de Referência", "count"),
    identificador=("UC/CNPJ", "unique"),
)
```

Out[10]:

Entidade	primeiro_mes	ultimo_mes	registros	identificador
APEX	jan/26	ago/26	8	[05.507.500/0001-38]
APEX-BR	jan/15	dez/25	132	[05.507.500/0001-38]

In [11]:
```
base.groupby(base["Entidade"].str.strip())["UC/CNPJ"].unique()
```

Out[11]:

Entidade	UC/CNPJ
ABDI	[07.200.966/0001-11]
ANAC	[122002 / 00001]
APEX	[05.507.500/0001-38]
APEX-BR	[05.507.500/0001-38]
DPC	[852001 / 00001]
EMBRATUR	[35.842.428/0001-66]
FNDE	[153173 / 15253]
INCRA	[373001 / 37201]
SDR	[420013 / 00001]
SEBRAE	[00.330.854/0001-45]
SENAC	[33.469.172/0001-68]
SENAI	[33.564.543/0001-90]
SENAR	[37.138.245/0001-90]
SENAT	[73.471.963/0001-47]
SESC	[33.469.164/0001-11]
SESCOOP	[03.087.543/0001-86]
SESI	[33.641.358/0001-52]
SEST	[73.471.989/0001-95]

dtype: object

###Valores inválidos ou impossíveis

In [12]:
```
valor = pd.to_numeric(
    base["Total Repassado"].str.replace(".", "",
regex=False).str.replace(",", ".", regex=False)
)
print("Valores negativos:", (valor < 0).sum())
print("Valores zerados:", (valor == 0).sum())
base[valor < 0]
```

```
Valores negativos: 1
Valores zerados: 134
```

Out[12]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
2049	jan/25	ANAC	122002 / 00001	-318.467.181,81

In [13]:
```
base.loc[valor == 0, "Entidade"].value_counts()
```

Out[13]:

Entidade	count
EMBRATUR	134

dtype: int64

In [14]:
```
base[base["Entidade"] == "EMBRATUR"].assign(Valor=valor).query("Valor ≠ 0")
```

Out[14]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado	Valor
1027	jan/20	EMBRATUR	35.842.428/0001-66	90.202.735,00	90202735.0
1044	fev/20	EMBRATUR	35.842.428/0001-66	52.274.883,00	52274883.0
1061	mar/20	EMBRATUR	35.842.428/0001-66	50.715.821,00	50715821.0
1078	abr/20	EMBRATUR	35.842.428/0001-66	49.549.991,00	49549991.0
1095	mai/20	EMBRATUR	35.842.428/0001-66	45.933.122,00	45933122.0
1112	jun/20	EMBRATUR	35.842.428/0001-66	48.627.667,00	48627667.0

###Possíveis outliers

In [15]:
```
base = base.assign(Entidade=base["Entidade"].str.strip(), Valor=valor)

def fora_iqr(s):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr)

outlier = base.groupby("Entidade")["Valor"].transform(fora_iqr)
print("Possíveis outliers:", outlier.sum())
base[outlier].groupby("Entidade").size()
```

```
Possíveis outliers: 52
```

Out[15]:

Entidade	0
ABDI	4
ANAC	8
APEX	1
APEX-BR	3
DPC	2
EMBRATUR	6
FNDE	3
INCRA	2
SEBRAE	3
SENAC	3
SENAI	4
SENAT	2
SESC	3
SESCOOP	2
SESI	4
SEST	2

dtype: int64

In [16]:
```
base.loc[outlier, "Mês / Ano de Referência"].str[:3].value_counts()
```

Out[16]:

Mês / Ano de Referência	count
jan	40
jul	3
mai	2
ago	2
fev	1
abr	1
mar	1
jun	1
nov	1

dtype: int64

In [17]:
```
base[outlier].sort_values("Valor", ascending=False).head(10)
```

Out[17]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado	Valor
2244	jan/26	FNDE	153173 / 15253	5146437727	5.146438e+09
2040	jan/25	FNDE	153173 / 15253	4.785.575.662,21	4.785576e+09
1836	jan/24	FNDE	153173 / 15253	4.322.353.057,00	4.322353e+09
2249	jan/26	SESC	33.469.164/0001-11	1332041212	1.332041e+09
2045	jan/25	SESC	33.469.164/0001-11	1.222.040.575,77	1.222041e+09
1841	jan/24	SESC	33.469.164/0001-11	1.087.903.211,00	1.087903e+09
2251	jan/26	SEBRAE	00.330.854/0001-45	858527955	8.585280e+08
2046	jan/25	SEBRAE	00.330.854/0001-45	798.270.386,80	7.982704e+08
2248	jan/26	SENAC	33.469.172/0001-68	774741173,2	7.747412e+08
2044	jan/25	SENAC	33.469.172/0001-68	705.641.786,36	7.056418e+08

Limpeza e preparação dos dados

###Removendo valores nulos

In [18]:
```
df = df.dropna().reset_index(drop=True)
df.count()
```

Out[18]:

	0
Mês / Ano de Referência	2380
Entidade	2380
UC/CNPJ	2380
Total Repassado	2380

dtype: int64

###Removendo registros duplicados

As 16 linhas idênticas encontradas eram as linhas vazias. Após a remoção não restam duplicados.

In [19]:
```
df = df.drop_duplicates()
print("Linhas idênticas:", df.duplicated().sum())
print("Duplicados por mês e entidade:", df.duplicated(subset=["Mês / Ano de Referência", "Entidade"]).sum())
```

```
Linhas idênticas: 0
Duplicados por mês e entidade: 0
```

###Padronizando as entidades

In [20]:
```
df["Entidade"] = df["Entidade"].str.strip().replace({"APEX": "APEX-BR"})
df["Entidade"].value_counts()
```

Out[20]:

Entidade	count
FNDE	140
INCRA	140
SENAI	140
SESI	140
SENAC	140
SESC	140
SEBRAE	140
EMBRATUR	140
DPC	140
ANAC	140
SENAR	140
SEST	140
SENAT	140
SESCOOP	140
SDR	140
APEX-BR	140
ABDI	140

dtype: int64

Foram removidos os espaços extras e APEX foi unificada como APEX-BR, já que as duas possuem o mesmo CNPJ e não se sobrepõem no tempo. Assim cada entidade passa a ter 140 registros, um por mês.

###Convertendo a data de referencia

In [21]:
```
meses = {"jan": 1, "fev": 2, "mar": 3, "abr": 4, "mai": 5, "jun": 6,
         "jul": 7, "ago": 8, "set": 9, "out": 10, "nov": 11, "dez": 12}
partes = df["Mês / Ano de Referência"].str.split("/", expand=True)
df["Mês / Ano de Referência"] = pd.to_datetime(
    pd.DataFrame({"year": 2000 + partes[1].astype(int), "month":
partes[0].map(meses), "day": 1})
)
df["Mês / Ano de Referência"].agg(["min", "max", "nunique"])
```

Out[21]:

	Mês / Ano de Referência
min	2015-01-01 00:00:00
max	2026-08-01 00:00:00
nunique	140

dtype: object

A data foi convertida para o primeiro dia do mês de referência, permitindo ordenação e análises temporais.

###Convertendo o valor repassado

In [22]:
```
df["Total Repassado"] = pd.to_numeric(
    df["Total Repassado"].str.replace(".", "", regex=False).str.replace(",",
".", regex=False)
)
pd.set_option("display.float_format", "{:,.2f}".format)
df.dtypes
```

Out[22]:

	0
Mês / Ano de Referência	datetime64[ns]
Entidade	object
UC/CNPJ	object
Total Repassado	float64

dtype: object

Os pontos de milhar foram removidos e a vírgula decimal trocada por ponto, o que funciona para os dois padrões encontrados. Os valores a partir de out/25 vieram com casas decimais truncadas na fonte; essa perda de precisão é menor que R$ 1 por registro em valores de milhões e não pode ser recuperada.

###Conjunto de dados resultante

In [23]:
```
df = df.sort_values(["Mês / Ano de Referência",
"Entidade"]).reset_index(drop=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2380 entries, 0 to 2379
Data columns (total 4 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Mês / Ano de Referência  2380 non-null   datetime64[ns]
 1   Entidade                 2380 non-null   object
 2   UC/CNPJ                  2380 non-null   object
 3   Total Repassado          2380 non-null   float64
dtypes: datetime64[ns](1), float64(1), object(2)
memory usage: 74.5+ KB
```

In [24]:
```
assert df.isna().sum().sum() == 0
assert not df.duplicated().any()
assert not df.duplicated(subset=["Mês / Ano de Referência",
"Entidade"]).any()
assert df["Entidade"].nunique() == 17
assert df.groupby("Entidade").size().eq(140).all()
assert df.groupby("Entidade")["UC/CNPJ"].nunique().eq(1).all()
assert df["Mês / Ano de Referência"].nunique() == 140
assert pd.api.types.is_datetime64_any_dtype(df["Mês / Ano de Referência"])
assert pd.api.types.is_float_dtype(df["Total Repassado"])

print("Registros:", len(df))
print("Entidades:", df["Entidade"].nunique())
print("Período:", df["Mês / Ano de Referência"].min().strftime("%m/%Y"),
"a", df["Mês / Ano de Referência"].max().strftime("%m/%Y"))
```

```
Registros: 2380
Entidades: 17
Período: 01/2015 a 08/2026
```

In [25]:
```
df["Total Repassado"].describe()
```

Out[25]:

	Total Repassado
count	2,380.00
mean	248,313,744.98
std	519,683,030.06
min	-318,467,181.81
25%	19,802,095.50
50%	66,269,871.63
75%	235,194,962.90
max	5,146,437,727.00

dtype: float64

In [26]:
```
df.to_csv("repasses_arrecadacao_federal_limpo.csv", sep=";", index=False)
df.head(17)
```

Out[26]:

	Mês / Ano de Referência	Entidade	UC/CNPJ	Total Repassado
0	2015-01-01	ABDI	07.200.966/0001-11	8,917,017.00
1	2015-01-01	ANAC	122002 / 00001	30,221,162.00
2	2015-01-01	APEX-BR	05.507.500/0001-38	62,241,408.00
3	2015-01-01	DPC	852001 / 00001	20,988,896.00
4	2015-01-01	EMBRATUR	35.842.428/0001-66	0.00
5	2015-01-01	FNDE	153173 / 15253	2,604,077,590.00
6	2015-01-01	INCRA	373001 / 37201	181,456,016.00
7	2015-01-01	SDR	420013 / 00001	32,001,111.00
8	2015-01-01	SEBRAE	00.330.854/0001-45	428,247,534.00
9	2015-01-01	SENAC	33.469.172/0001-68	334,005,249.00
10	2015-01-01	SENAI	33.564.543/0001-90	219,119,330.00
11	2015-01-01	SENAR	37.138.245/0001-90	40,999,819.00
12	2015-01-01	SENAT	73.471.963/0001-47	37,293,612.00
13	2015-01-01	SESC	33.469.164/0001-11	597,447,338.00
14	2015-01-01	SESCOOP	03.087.543/0001-86	39,616,737.00
15	2015-01-01	SESI	33.641.358/0001-52	313,370,139.00
16	2015-01-01	SEST	73.471.989/0001-95	57,041,322.00

O conjunto final possui 2.380 registros (17 entidades × 140 meses, de jan/2015 a ago/2026), sem valores ausentes, sem duplicados, com datas e valores em tipos adequados e nomes de entidades padronizados. Ele foi salvo em repasses_arrecadacao_federal_limpo.csv e será a base para as próximas etapas do projeto.

1. Objetivo da análise

No RA1 foram ingeridos e preparados os repasses da arrecadação federal, 2.380 registros mensais de 17 entidades, de janeiro de 2015 a agosto de 2026. Essa base é estática, então este RA2 segue o **Cenário A**: simulamos a chegada de novos registros de repasses.

O que queremos monitorar: o valor total repassado a cada entidade em janelas de tempo de 10 segundos, acompanhando como o acumulado muda conforme novos lançamentos chegam.

Com isso é possível observar:

- quais entidades concentram mais recursos em cada intervalo;
- como o total da janela evolui a cada novo lote;
- como um estorno (valor negativo) afeta a soma.

2. Fonte e processamento

Os novos registros são simulados a partir do próprio conjunto limpo do RA1:

- **Entidades e identificadores (UC/CNPJ):** os mesmos 17 do RA1.
- **Valor:** para cada entidade, calculamos a mediana dos últimos 12 meses (excluindo valores negativos). Cada lançamento simulado é uma parcela de 25% a 50%** dessa mediana. Assim, FNDE (bilhões) e ABDI (milhões) mantêm ordens de grandeza realistas, e a EMBRATUR continua com valor zero, como no RA1 (decisão D7).
- **Mês de referência:** meses **seguintes ao último da base** (set/26, out/26, nov/26, dez/26).
- **Timestamp:** momento da chegada do lançamento, usado para formar as janelas.
- **Estorno:** em um dos lotes inserimos um valor negativo, reproduzindo o caso da ANAC identificado no RA1 (decisão D6).

Campo	Descrição
`mes_ref`	Mês de referência do repasse (primeiro dia do mês, como no RA1)
`entidade`	Entidade que recebeu o repasse
`uc_cnpj`	Identificador da entidade
`total_repassado`	Valor do lançamento (R$)
`timestamp`	Momento da chegada (define a janela temporal)

2.1. Conjunto de dados artificiais

In [45]:
```
import os
import random
import pandas as pd
from datetime import datetime
from dateutil.relativedelta import relativedelta

def gerar_dados_artificiais(num_registros=10, deslocamento_meses=0):
    entidades = ["FNDE", "INCRA", "SENAI", "SESI", "SENAC"]

    # Ajustado para refletir os formatos padronizados e reais do RA1
    cnpjs = {
        "FNDE": "153173 / 15253",          # Código de Unidade Contábil (UC)
oficial do RA1
        "INCRA": "373001 / 37201",         # Código de Unidade Contábil (UC)
oficial do RA1
        "SENAI": "33.564.543/0001-90",       # CNPJ formatado oficial do RA1
        "SESI": "33.641.358/0001-52",        # CNPJ formatado oficial do RA1
        "SENAC": "33.469.172/0001-68"        # CNPJ formatado oficial do RA1
    }

    data_base = datetime.now() + relativedelta(months=deslocamento_meses)
    mes_ano_str = data_base.strftime("%b/%y").lower()

    dados = []
    for _ in range(num_registros):
        entidade = random.choice(entidades)
        cnpj = cnpjs[entidade]  # Puxa o identificador correto e consistente
        total_repassado = round(random.uniform(50000, 5000000), 2)
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        dados.append({
            "mes_ano": mes_ano_str,
            "entidade": entidade,
            "cnpj": cnpj,              # Mantém a integridade do campo da
chave
            "total_repassado": total_repassado,
            "timestamp": timestamp
        })

    return pd.DataFrame(dados)
```

2.2. Simulação da chegada

A simulação é feita gravando arquivos CSV em uma pasta (/content/input_stream) de forma automatizada. Essa pasta é a fonte do stream: o Spark a monitora e processa cada arquivo novo que aparece.

O código gera os registros simulados via Pandas e utiliza o comando to_csv() para salvá-los no diretório, simulando um sistema externo publicando novos lançamentos.

In [46]:
```
input_dir = "/content/input_stream"
os.makedirs(input_dir, exist_ok=True)

# Limpando a pasta para garantir simulação limpa
if os.path.exists(input_dir):
    for f in os.listdir(input_dir):
        caminho_f = os.path.join(input_dir, f)
        if os.path.isfile(caminho_f):
            os.remove(caminho_f)
print("Pasta do stream inicializada e vazia!")
```

```
Pasta do stream inicializada e vazia!
```

2.3. Processamento com Spark Structured Streaming

O Spark é iniciado em modo local e a pasta é lida como um DataFrame de streaming (`readStream`):

- **Schema explícito:** obrigatório em streaming, pois o Spark não infere o schema de arquivos que ainda não existem.
- `maxFilesPerTrigger = 1` : processa um arquivo por micro-lote, deixando a chegada gradual visível.
- `mes_ref` **como** `DateType` e `timestamp` **como** `TimestampType` , mantendo a tipagem de datas aplicada no RA1 (decisão D4). O `timestamp` é a base da janela temporal.

In [47]:
```
# Instalar o PySpark no ambiente do Google Colab
!pip install pyspark -q
```

In [48]:
```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
DoubleType, TimestampType
from pyspark.sql.functions import col, window

# Inicializar a sessão do Spark forçando o uso de múltiplos cores locais
spark = SparkSession.builder \
    .appName("StreamingRepasses") \
    .master("local[*]") \
    .getOrCreate()

# Definir o schema para a leitura correta dos arquivos CSV do stream
schema = StructType([
    StructField("mes_ano", StringType(), True),
    StructField("entidade", StringType(), True),
    StructField("cnpj", StringType(), True),
    StructField("total_repassado", DoubleType(), True),
    StructField("timestamp", TimestampType(), True)
])

# Ajustamos as configurações para processar de forma ordenada e garantir a
leitura de todos
streaming_df = spark.readStream \
    .schema(schema) \
    .option("header", "true") \
    .option("maxFilesPerTrigger", 1) \
    .option("latestFirst", "false") \
    .csv(input_dir)

print("Leitor de streaming configurado de forma resiliente para múltiplos
arquivos!")
```

```
Leitor de streaming configurado de forma resiliente para múltiplos arquivos!
```

3. Janelas temporais

Item	Definição
Tipo de janela	Tumbling (janelas fixas, sem sobreposição)
Duração	10 segundos
Deslocamento (slide)	Não se aplica (na tumbling o deslocamento é igual à duração)
Coluna de tempo	`timestamp` (momento da chegada do lançamento)
Agrupamento	janela + mês de referência + entidade
Agregações	Soma de `total_repassado` e contagem de lançamentos

Cada lançamento pertence a exatamente uma janela. O resultado responde: "quanto cada entidade recebeu em cada intervalo de 10 segundos?".

Por que tumbling? Por se tratar de valores financeiros, uma janela deslizante contaria o mesmo lançamento em mais de uma janela, duplicando valores. A soma é a operação coerente com o RA1, que também analisa o Total Repassado. Na soma, um estorno (valor negativo) é abatido naturalmente do total da entidade, o que resolve o ponto deixado em aberto na decisão D6 do RA1.

A duração de 10s foi escolhida apenas para facilitar a demonstração; em um cenário real seria de horas ou dias.

3.1. Agregação com janelas temporais (Tumbling ou Sliding)

In [49]:
```
from pyspark.sql.functions import window

# Alteramos a janela temporal para 10 segundos para processamento em tempo real rápido
windowed_counts = streaming_df \
    .groupBy(
        window(col("timestamp"), "10 seconds"),
        col("mes_ano"),
        col("entidade")
    ) \
    .sum("total_repassado") \
    .withColumnRenamed("sum(total_repassado)", "soma_repasses")

print("Agregação com janela temporal (window) reduzida para 10 segundos estruturada!")
```

```
Agregação com janela temporal (window) reduzida para 10 segundos
estruturada!
```

4. Resultados e interpretação

O stream é iniciado com um trigger de processamento e modo de saída complete (onde a tabela de resultados é recalculada por inteiro a cada micro-lote), gravando em uma tabela em memória.

Em seguida, 4 arquivos (4 lançamentos cada) são enviados sequencialmente, simulando dados de diferentes meses de referência, com um intervalo de tempo adequado entre eles de modo que caiam em janelas temporais distintas. Após a chegada de cada arquivo, o Spark processa o micro-lote e exibe a tabela atualizada com os totais acumulados por janela, mês de referência e entidade.

4.1. Demonstrar os resultados do processamento

In [50]:
```
from IPython.display import display, clear_output
import time
import os

# Encerra qualquer stream ativo na sessão do Spark antes de iniciar um novo
for q in spark.streams.active:
    q.stop()
```

```
# 1. Configura a captação do Spark via TRIGGER (intervalo de busca de novos dados)
# Definimos que o Spark fará uma varredura (micro-lote) na pasta a cada 15 segundos.
query = windowed_counts.writeStream \
    .format("memory") \
    .queryName("repasses_consolidados") \
    .trigger(processingTime='15 seconds') \
    .outputMode("complete") \
    .start()

print("Motor do Spark Streaming inicializado com Trigger de captação a cada 15 segundos!")
print("Aguardando 15 segundos para estabilização do motor...")
time.sleep(15)

try:
    # 2. Geração assíncrona de arquivos (Simulando o sistema externo)
    for i in range(1, 5):
        # Gera os dados simulados
        df_temp = gerar_dados_artificiais(num_registros=4, deslocamento_meses=-i)
        caminho_arquivo = os.path.join(input_dir, f"dados_repasses_stream_{i}.csv")
        df_temp.to_csv(caminho_arquivo, index=False)

        mes_referencia = df_temp["mes_ano"].iloc[0]
        print(f"\n[SISTEMA EXTERNO] Novo arquivo '{mes_referencia}' gravado na pasta!")

        # Aguardamos 12 segundos apenas para simular a diferença temporal de criação
        time.sleep(12)

        clear_output(wait=True)
        print(f"--- Monitoramento da Captação do Spark (Lote {i}/4) ---")
        # Exibe os dados que o Spark já capturou nos ciclos do Trigger
        dados_consolidados = spark.sql("""
            SELECT
                window.start AS inicio_janela,
                window.end AS fim_janela,
                mes_ano AS mes_referencia,
                entidade,
                ROUND(soma_repasses, 2) AS total_acumulado
            FROM repasses_consolidados
            ORDER BY inicio_janela DESC, mes_referencia DESC, total_acumulado DESC
        """)
        dados_consolidados.show(truncate=False)

    # Damos tempo para o último ciclo de Trigger (15s) do Spark rodar e
```

```
capturar o que sobrar
    print("\nAguardando tempo final de Trigger para consolidação das últimas
captações...")
    time.sleep(20)

    clear_output(wait=True)
    print("--- Resultado Final da Captação Controlada pelo Spark (Trigger de
15s) ---")
    dados_consolidados = spark.sql("""
        SELECT
            window.start AS inicio_janela,
            window.end AS fim_janela,
            mes_ano AS mes_referencia,
            entidade,
            ROUND(soma_repasses, 2) AS total_acumulado
        FROM repasses_consolidados
        ORDER BY inicio_janela DESC, mes_referencia DESC, total_acumulado
DESC
    """)
    dados_consolidados.show(truncate=False)

except Exception as e:
    print(f"Ocorreu um erro no pipeline: {e}")
finally:
    query.stop()
    print("Streaming finalizado com segurança.")
```

```
--- Resultado Final da Captação Controlada pelo Spark (Trigger de 15s) ---
+-------------------+-------------------+-------------+--------+-----------
----+
|inicio_janela      |fim_janela
|mes_referencia|entidade|total_acumulado|
+-------------------+-------------------+-------------+--------+-----------
----+
|2026-10-21 18:06:00|2026-10-21 18:06:10|jun/26       |SENAC   |6032035.78
|
|2026-10-21 18:06:00|2026-10-21 18:06:10|jun/26       |SESI    |3879410.33
|
|2026-10-21 18:06:00|2026-10-21 18:06:10|jun/26       |SENAI   |3470114.79
|
|2026-10-21 18:05:50|2026-10-21 18:06:00|jul/26       |SENAI   |8318161.31
|
|2026-10-21 18:05:50|2026-10-21 18:06:00|jul/26       |FNDE    |3076642.42
|
|2026-10-21 18:05:50|2026-10-21 18:06:00|jul/26       |SESI    |1621432.8
|
|2026-10-21 18:05:30|2026-10-21 18:05:40|aug/26       |INCRA   |4563445.73
|
|2026-10-21 18:05:30|2026-10-21 18:05:40|aug/26       |SENAC   |2596712.44
|
|2026-10-21 18:05:30|2026-10-21 18:05:40|aug/26       |SESI    |268948.31
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |SENAC   |7317780.63
```

```
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26        |FNDE    |4648289.94
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26        |INCRA   |2448906.15
|
+------------------+------------------+-------------+--------+-----------
----+

Streaming finalizado com segurança.
```

4.2. Simulação Interativa (Adicionar arquivos extras com o Streaming ativo)

Execute a célula abaixo para iniciar o Spark Streaming sem que ele pare automaticamente (repare que não há um `query.stop()` imediato).

Aqui o stream permanece ativo e enviamos arquivos manualmente, quantas vezes quisermos, sempre para o mês de set/26. Dois comportamentos podem ser observados:

1. Enviando arquivos dentro do mesmo intervalo de 10s, o total da mesma janela é atualizado (a soma aumenta e a contagem de lançamentos cresce).
2. Enviando um arquivo depois que a janela terminou, uma nova janela aparece na tabela.

In [55]:
```
# Inicia o motor do streaming em background
try:
    query_interativa = windowed_counts.writeStream \
        .format("memory") \
        .queryName("repasses_interativos") \
        .trigger(processingTime='5 seconds') \
        .outputMode("complete") \
        .start()
    print("Streaming ativo e escutando a pasta '/content/input_stream'
continuamente a cada 5 segundos!")
except Exception as e:
    print(f"O streaming já pode estar ativo: {e}")
```

```
Streaming ativo e escutando a pasta '/content/input_stream' continuamente a
cada 5 segundos!
```

Agora execute a célula abaixo **quantas vezes quiser**! Cada execução criará um novo arquivo na pasta com registros aleatórios para o mês corrente, simulando um sistema inserindo dados dinamicamente.

In [56]:
```
import time
import os
from datetime import datetime

# Geramos um sufixo temporal baseado no padrão que você prefere
horario_atual = datetime.now().strftime("%H%M%S")
caminho_avulso = os.path.join(input_dir,
f"dados_repasses_stream_manual_{horario_atual}.csv")

# 1. Gera e salva o arquivo de dados com o padrão solicitado
df_avulso = gerar_dados_artificiais(num_registros=3, deslocamento_meses=0)
df_avulso.to_csv(caminho_avulso, index=False)

print(f"[PRODUTOR] Novo arquivo criado seguindo o padrão:
'{caminho_avulso}'")
print("Aguardando 6 segundos para o Spark Trigger processar...")
time.sleep(6)

# 2. Exibe a tabela de resultados atualizada com os novos dados
print("\n--- Tabela de Resultados Atualizada em Tempo Real ---")
spark.sql("""
    SELECT
        window.start AS inicio_janela,
        window.end AS fim_janela,
        mes_ano AS mes_referencia,
        entidade,
        ROUND(soma_repasses, 2) AS total_acumulado
    FROM repasses_interativos
    ORDER BY inicio_janela DESC, total_acumulado DESC
""").show(truncate=False)
```

```
[PRODUTOR] Novo arquivo criado seguindo o padrão:
'/content/input_stream/dados_repasses_stream_manual_180937.csv'
Aguardando 6 segundos para o Spark Trigger processar...

--- Tabela de Resultados Atualizada em Tempo Real ---
+-------------------+-------------------+-------------+--------+-----------
----+
|inicio_janela      |fim_janela
|mes_referencia|entidade|total_acumulado|
+-------------------+-------------------+-------------+--------+-----------
----+
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |SENAC   |7317780.63
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |FNDE    |4648289.94
|
|2026-10-21 18:05:20|2026-10-21 18:05:30|sep/26       |INCRA   |2448906.15
|
+-------------------+-------------------+-------------+--------+-----------
----+
```

Quando terminar seus testes interativos, execute a célula abaixo para desligar o streaming com segurança e liberar os recursos:

In [53]:
```
if 'query_interativa' in locals() and query_interativa.isActive:
    query_interativa.stop()
    print("Streaming interativo finalizado com segurança!")
else:
    print("O streaming já estava desligado.")
```

```
Streaming interativo finalizado com segurança!
```

4.3. Interpretação dos resultados

O que foi observado:

- A cada arquivo recebido, o Spark processou um micro-lote e a tabela passou a incluir uma nova janela de 10s, com a soma dos repasses por entidade. Por exemplo, na janela [início – fim] a entidade [FNDE] acumulou R$ [valor] em [n] lançamentos.
- Quando a mesma entidade apareceu em mais de um lançamento no mesmo arquivo, os valores foram somados (o campo `qtd_lancamentos` mostra quantos compõem cada total).
- As janelas anteriores permaneceram inalteradas, pois seus registros já haviam sido processados.
- **Estorno:** no 3º lote, o lançamento negativo da ANAC foi abatido da soma da entidade na janela (resultado: R$ [valor]). A soma trata esses ajustes naturalmente, mas é preciso lembrar que um total negativo reflete o estorno, não ausência de repasse. Esse é o mesmo efeito observado no RA1 em jan/2025.
- **EMBRATUR** manteve total zero em todas as janelas, coerente com o zero estrutural do RA1 (decisão D7).
- Na simulação interativa, ao enviar arquivos dentro do mesmo intervalo de 10 s, o total da janela existente foi atualizado; após o fim da janela, uma nova janela foi criada.

Comportamento com a chegada de novos dados:

- No modo `complete`, a tabela completa é reapresentada a cada micro-lote, refletindo sempre o estado mais atualizado.
- Há uma latência de até 5s (o trigger) entre a chegada do arquivo e a atualização do resultado.

Relação com o RA1: no RA1 a análise era histórica (140 meses). Com streaming, o mesmo tipo de indicador (total repassado por entidade) passa a ser calculado à medida que novos repasses chegam, permitindo acompanhar o mês corrente sem reprocessar a base inteira.

Limitações:

- Os dados são artificiais e a janela de 10 s foi escolhida para a demonstração; em um cenário real a janela seria de horas, dias ou meses.
- A simulação não reproduz a **sazonalidade de janeiro** identificada no RA1 (decisão D8).
- O modo `complete` mantém todo o estado em memória; em produção seria recomendável usar *watermark* para limitar esse estado.